

Foundry: Conceptual Design

1. Introduction

Foundry aims to be a highly flexible and interconnected workspace designed for organizing diverse types of information. It combines the freeform nature of document editing with the structured power of databases, allowing users to create, connect, and customize their information hubs for various tasks, from project management and note-taking to personal knowledge management and collaborative whiteboarding.

2. Core Value Proposition

Foundry empowers users to build their own tools for thinking and organizing. Instead of forcing information into predefined formats, Foundry provides versatile building blocks that can be combined and customized to fit the specific needs of the user and the task at hand.

3. Core Features

3.1. Block-Based Editor

The foundation of Foundry is its block editor. Every piece of content, from a paragraph of text to an image or a database, is a distinct "block."

- **Content Types:** Supports a wide range of blocks:
 - **Basic:** Text (with rich formatting), Headings, Bulleted Lists, Numbered Lists, To-do Lists, Toggles, Quotes, Dividers, Callouts.
 - **Media:** Images (upload, link, Unsplash integration), Video Embeds (YouTube, Vimeo, etc.), Audio Files, General File Uploads.
 - **Code:** Code blocks with syntax highlighting for various languages.
 - **Embeds:** Embed content from other web services (e.g., Google Maps, Figma, Miro, Twitter).
 - **Advanced:** Math Equations (LaTeX), Breadcrumbs, Table of Contents.
 - **Database Views:** Inline or full-page views of databases (detailed below).
- **Manipulation:** Blocks can be easily dragged and dropped to rearrange content, indented to create hierarchy, and transformed into other compatible block types.
- **Collaboration:** Real-time multi-user editing with presence indicators and commenting on blocks.

3.2. Powerful Databases

Databases are structured containers for information within Foundry. They are more than simple tables; they are collections of pages, each with customizable properties.

- **Creation:** Databases can be created inline on a page or as dedicated full-page databases.
- **Properties (Columns):** Users define custom properties for database entries (pages):
 - **Types:** Text, Number, Select (dropdown), Multi-select, Date, Person (assigning users), Files & Media, Checkbox, URL, Email, Phone, Formula (perform calculations based on other properties), Relation (link to entries in other databases), Rollup (aggregate data from related entries).
- **Views:** The key power lies in viewing database content in multiple ways without duplicating data:
 - **Table View:** Traditional spreadsheet-like view. Sortable, filterable, groupable by properties. Property visibility can be toggled.
 - **Board View (Kanban):** Cards grouped into columns based on a 'Select' or 'Person' property. Ideal for tracking progress (e.g., To Do, In Progress, Done). Cards are draggable between columns.
 - **List View:** A simplified, clean vertical list of database pages. Customizable property visibility.
 - **Calendar View:** Entries displayed on a calendar based on a 'Date' property. Useful for deadlines, events, content schedules.
 - **Gallery View:** Visually focused view displaying pages as cards with a preview (e.g., cover image, content snippet). Customizable card size and visible properties.
- **Filtering & Sorting:** Robust filtering and sorting options applicable across all views. Users can save specific view configurations (e.g., "My Open Tasks," "This Month's Events").
- **Database Templates:** Define templates for pages created *within* a specific database, pre-filling properties or page content.

3.3. Interlinked Pages (Bi-directional Linking)

Foundry facilitates the creation of interconnected knowledge networks.

- **Page Linking:** Easily create links to other pages using @ mentions, which searches page titles.
- **Backlinks:** Automatically displays links to the current page from other pages, providing context and discoverability.
- **Relations (Database Property):** Explicitly link database entries together (e.g., linking Tasks to Projects, Meeting Notes to Attendees).

3.4. Templates

Templates streamline the creation of new pages and structures.

- **Pre-built Templates:** Offer a gallery of templates for common use cases (e.g., Meeting Notes, Project Brief, Content Calendar, Habit Tracker, Travel Planner).
- **Custom Templates:** Users can design their own pages or database structures and save them as templates for reuse.
- **Template Button:** A special block that, when clicked, generates a predefined structure of blocks or a new page based on a template.

3.5. High Customization

Flexibility is central to Foundry's design.

- **Page Layout:** Options for full-width pages, customizable page icons and cover images.
- **Block Arrangement:** Drag-and-drop blocks side-by-side to create columns.
- **Database Views:** Customize visible properties, card previews, grouping, sorting, and filtering for each view.
- **Formulas:** Create powerful calculations and data manipulations within databases.
- **API (Potential):** A future API could allow for integrations and automation.

4. Use Cases

- **Whiteboards:** Use freeform text, drawings (via embeds or future native block), images, and links for brainstorming sessions.
- **Meeting Notes:** Create templates with sections for attendees, agenda, discussion points, and action items (linked to a Tasks database).
- **Travel Details:** Build a travel dashboard with pages for destinations, linked databases for flights (with dates, costs), accommodations (with addresses, booking links), and packing lists (using to-do blocks).
- **Project Management:** Use databases with board views for task tracking, calendar views for deadlines, and linked pages for project briefs and documentation.
- **Personal Wiki/Knowledge Base:** Interlink pages extensively to create a personal repository of notes, research, and ideas.

5. Conceptual Architecture (High-Level)

- **Frontend:** Likely a web-based application built with a modern JavaScript framework (e.g., React, Vue) for a dynamic and responsive block editor experience. Real-time capabilities using WebSockets.
- **Backend:** Needs to handle user authentication, data storage (likely a NoSQL or document database suitable for flexible schemas), real-time synchronization, search indexing, and potentially API management.
- **Data Model:** A block-centric model where pages are collections of blocks, and

databases are special types of blocks containing structured page entries with properties. Relationships need careful modeling.

6. UI/UX Considerations

- **Clean & Minimal:** Default interface should be unobtrusive, focusing on the content.
- **Intuitive Editing:** Block manipulation (adding, moving, transforming) should feel natural and discoverable (e.g., using / commands, drag handles).
- **Responsive Design:** Ensure usability across desktop, tablet, and mobile devices.
- **Performance:** Critical for handling large pages and databases smoothly, especially with real-time collaboration.

7. Future Directions

- Native Mobile Apps
- Offline Mode
- Advanced Permissions and Sharing Controls
- Version History
- Public API for Integrations
- More Block Types (e.g., Native Drawing, Mind Maps)
- AI Features (Summarization, Content Generation)